Playwright Introduction

1. What is Playwright?

- Playwright is an **open-source automation tool** developed by **Microsoft** for web browser testing.
- It allows you to **automate browsers** and perform **end-to-end testing** of web applications.
- Apart from UI automation, it also supports **API testing** with its built-in API testing capabilities.
- Available as an **open-source Python library** (and other languages).
- **First released:** 2020.

2. Key Features

Cross-capability

1. **Cross-browser** → Works with Chromium, Firefox, and WebKit.
2. **Cross-platform** → Runs on Windows, macOS, and Linux.
3. **Cross-language** → Supports Java, Python, JavaScript/TypeScript, and C#.

Testing Capabilities

- **Mobile Web Testing** → Can simulate mobile devices and provides experimental Android automation support.
- **API Testing** → Built-in API testing to test backend services.
- **Automatic Waiting** → Waits for elements to be ready before interacting with them, reducing flaky tests.
- **Handles Complex Elements** → Works with Shadow DOM and other tricky web components.
- **Parallel Execution** → Run multiple tests at the same time for faster execution.
- **Built-in Reporters** → HTML, JSON, JUnit, etc., plus third-party options like Allure.

3. Advanced Features

1. **Inspector** → Debug tests visually by highlighting click points and checking locators in real time.

2. **Code Generation (Codegen)** → Records user actions and generates scripts in any supported language.
3. **Tracing (Trace Viewer)** → Captures screenshots, videos, and step logs for easy debugging of test failures.

4. Why Use Playwright with Python?

1. **Popularity with Less Competition** → Python has a large market share, but fewer test automation professionals compared to Java/JS, meaning more opportunities.
2. **Industry Adoption** → Widely used in data-focused companies, startups, and AI-driven firms due to its rich ecosystem.
3. **Ease of Learning** → Simple syntax and beginner-friendly structure.
4. **Better Integration** → Works seamlessly with tools for API testing, Machine Learning, AI, and data handling.
5. **Future-Proof** → Growing role in test automation, API testing, and AI-powered testing.
6. **Strong Support** → Official Playwright documentation and Python community are robust and helpful.

Python Basics

- **Introduction to Python**
- **Setup Environment**
- **Comments**
- **Keywords**
- **Variables**

Python Programming

- Python is a **high-level, general-purpose programming language**.
- It is **open-source** (free to use and modify).
- It is **platform-independent** – runs on Windows, macOS, Linux, etc.

Why Python?

- Simple and easy to learn.
- Large community support.
- Works for web development, data science, automation, AI, and more.

Download and Install Python

- Go to python.org → Download → Install on your system.

PyCharm

- PyCharm is a **popular IDE** (Integrated Development Environment) for Python.
- Helps in writing, running, and debugging Python code easily.

Comments in Python

- Comments are **non-executable** text in code.
- Python **ignores comments** while running.
- Uses:
    - Explain code.
    - Make code easier to read.
    - Temporarily disable code during testing.
- **Single-line comment:** starts with #

```
# This is a comment
```

- **Multi-line comment:** use triple quotes (''' or """)

"""

This is a

multi-line comment

"""

Keywords

- **Reserved words** in Python that have special meaning.
- Examples: if, else, while, for, True, False, None, etc.
- You **cannot** use keywords as variable names.

Variables

- Containers for storing data values.
- No need to declare before using.
- Python decides the data type automatically.
- Variable names are **case-sensitive** (name and Name are different).

Example:

```
x = 10       # integer
name = "John" # string
```

Variable Naming Conventions

- Can be short (x, y) or descriptive (age, car_name).
- **Rules:**
    - Must start with a **letter** or **underscore**.
    - Cannot start with a number.
    - Only letters, numbers, and underscores are allowed.
    - Case-sensitive.
    - Cannot be a Python keyword.

Valid Examples:

```
myvar = "John"

my_var = "John"

_my_var = "John"

myVar = "John"

MYVAR = "John"

myvar2 = "John"
```

Invalid Examples:

```
2myvar = "John"   # starts with number

my-var = "John"   # contains hyphen

my var = "John"   # contains space
```

Dynamic Typing

- Python is a **dynamically typed language**.
- You don’t need to declare a variable type—it is decided at runtime.

Example:

```
x = 10       # int

x = "Hello"   # now a string
```

Python Basics

- **Deleting Variables**
- **Data Types**
- **Operators**
- **Concatenation**
- **User Input**
- **Type Conversion/Type Casting**

Deleting Variables

- Use del to remove a variable from memory.
- Once deleted, you can't use it until recreated.

```
a = 100
b = 200
del a
print(b)  # Works → 200
```

Basic Data Types

- **int** → whole numbers (10, -5)
- **float** → decimal numbers (3.14, -0.5)
- **str** → text/strings ("Hello", 'Python')
- **bool** → True or False

```
x = 100       # int
y = 10.5      # float
s = "Python"  # str
b = True      # bool
print(type(x))  # <class 'int'>
```

Operators

- **Arithmetic:** +, -, *, /, %, ** (power), // (floor division)
- **Comparison:** ==, !=, >, <, >=, <= → returns True/False
- **Logical:** and, or, not → returns True/False

Concatenation

- + joins same-type values.
- Works for numbers, strings, and booleans (True=1, False=0).

```
print('Hello' + 'Python')  # HelloPython
print(True + 5)  # 6
```

Taking User Input

- input() → always returns string → convert if needed.

```
name = input("Enter name: ")
num1 = int(input("Enter number: "))
num2 = float(input("Enter decimal: "))
```

Type Casting

- Change data type: int(), float(), str().

```
x = "10"
y = int(x)  # "10" → 10
```

Python Conditional Statements

- Conditional statements are used to make decisions in Python.
- They allow the program to execute different blocks of code depending on a condition (True or False).

Indentation

Python relies on indentation (whitespace at the beginning of a line) to define scope in the code. Other programming languages often use curly-brackets for this purpose.

1. if Statement

The if statement executes a block of code only if the condition is **True**.

Syntax:

```
if condition:
    # code to execute if condition is True
```

Example:

```
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

Output:

```
You are eligible to vote.
```

2. if...else Statement

The else block runs when the if condition is **False**.

Syntax:

```
if condition:
    # code if condition is True
else:
    # code if condition is False
```

Example:

```
age = 16
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Output:

```
You are not eligible to vote.
```

3. if...elif...else Statement

Used when you have multiple conditions to check.

Syntax:

```
if condition1:
    # code if condition1 is True
elif condition2:
    # code if condition2 is True
else:
    # code if none of the conditions are True
```

Example:

```
marks = 75
if marks >= 90:
    print("Grade: A")
elif marks >= 75:
    print("Grade: B")
elif marks >= 50:
    print("Grade: C")
else:
    print("Fail")
```

Output:

Grade: B

Short Hand If

If you have only one statement to execute, you can put it on the same line as the if statement.

Example:

One line if statement:

```
if a > b: print("a is greater than b")
```

Short Hand If ... Else (Ternary Operator)

If you have only one statement to execute, one for if, and one for else, you can put it all on the same line:

Example:

One line if else statement:

```
a = 2
b = 330
print("A") if a > b else print("B")
```

You can also have multiple else statements on the same line:

Example:

One line if else statement, with 3 conditions:

```
a = 330
b = 330
print("A") if a > b else print("=") if a == b else print("B")
```

And:

The **and** keyword is a logical operator, and is used to combine conditional statements:

Example

Test if a is greater than b, AND if c is greater than a:

```
a = 200
b = 33
c = 500
if a > b and c > a:
  print("Both conditions are True")
```

Or

The or keyword is a logical operator, and is used to combine conditional statements:

Example

Test if a is greater than b, OR if a is greater than c:

```
a = 200
b = 33
c = 500
if a > b or a > c:
  print("At least one of the conditions is True")
```

Not

The not keyword is a logical operator, and is used to reverse the result of the conditional statement:

Example

Test if a is NOT greater than b:

```
a = 33
b = 200
if not a > b:
  print("a is NOT greater than b")
```

Nested If

You can have if statements inside if statements, this is called *nested* if statements.

Example

```
x = 41

if x > 10:
  print("Above ten,")
  if x > 20:
    print("and also above 20!")
  else:
    print("but not above 20.")
```

The pass Statement

if statements cannot be empty, but if you for some reason have an if statement with no content, put in the pass statement to avoid getting an error.

Example

```
a = 33
b = 200

if b > a:
  pass
```

Match, Range () function & Loops in Python

Python match Statement

- Used to choose one block of code to run from multiple options (like switch in other languages).
- **Why use it?** It's cleaner than writing many if...elif...else statements.
- **Syntax:**

```
match expression:
    case value1:
        # code block
    case value2:
        # code block
    case _:
        # default block (if no match)
```

- **How it works:**
    1. The expression is checked once.
    2. Compared with each case value.
    3. If a match is found, that block runs.

range() Function

- Generates a sequence of numbers.
- **Syntax:** range(start, stop, step)
- **Notes:**
    - start → beginning number (default 0)
    - stop → end number (exclusive)
    - step → how much to increase/decrease each time (default 1)

Looping Statements

1. While Loop – Runs **while** the condition is True.

```
while condition:
    # repeat code
```

2. For Loop – Runs a fixed number of times, often with range().

```
for variable in range(start, stop, step):
    # repeat code
```

Jumping Statements

- **break** → stop the loop immediately.
- **continue** → skip the current iteration, go to the next one.

Python Strings

Creating Strings

In Python, strings can be created in different ways:

- **Using double quotes**

```
name = "john"
grade = "B"
```

- **Using single quotes**

```
name = 'john'
grade = 'B'
```

- **Using str() constructor**

```
name = str()      # empty string
grade = str()     # empty string

name = str("David")
grade = str("B")

print(name)       # David
print(type(name))  # <class 'str'>
```

Note: Strings are always objects of the class str.

String Operators

- + → Concatenates (joins) strings
- * → Repeats a string multiple times

```
s = "welcome"
print(s + " programming")   # welcome programming
print(s * 3)               # welcomewelcomewelcome
```

Slicing Strings

- Strings can be sliced using indexes.
- Index starts from **0** (left → right).
- Negative index counts from **end** (right → left).

Example 1

```
mystr = "welcome"
print(mystr[1:3])    # el
print(mystr[:6])     # welcom
print(mystr[2:])     # lcome
print(mystr[1:-1])   # elcom
print(mystr[-5:-2])  # lco
```

Example 2

```
s = "abcdefghijklmno"
print(s[-4:])      # lmno
print(s[:-3])      # abcdefghijkl
print(s[-5:-2])    # klm
print(s[-8:-1:2])  # hjln
```

Explanation:

- s[-4:] → from 4th char from end (m) till end → "lmno"
- s[:-3] → from start till 3rd char from end (k excluded) → "abcdefghijkl"
- s[-5:-2] → from 5th char from end (l) till 2nd from end (excluded) → "klm"
- s[-8:-1:2] → from 8th char from end (g) to 2nd from end, step 2 → "hjln"

Reverse a String

Use a negative step (-1).

```
s = "Python"
print(s[::-1])  # nohtyP
```

Note: This creates a reversed copy. The original string remains unchanged.

Formatting Strings

- The **f-string** (Python 3.6+) is recommended.

```
age = 30
txt = f"My name is John, I am {age}"
print(txt)   # My name is John, I am 30


price = 59
print(f"The price is {price} dollars")       # The price is 59 dollars
print(f"The price is {price:.2f} dollars")   # The price is 59.00 dollars
print(f"The price is {20 * 59} dollars")     # The price is 1180 dollars
```

Old method: "Hello, {}".format(name) still works but f-strings are shorter.

Membership Operators

- in → Check if substring exists
- not in → Check if substring does not exist

```
s = "welcome"
print("come" in s)      # True
print("lome" in s)      # False
print("come" not in s)   # False
print("lome" not in s)   # True
```

Strings are Immutable

- **Mutable object** → Can be changed after creation (list, dict, set).
- **Immutable object** → Cannot be changed after creation (int, float, tuple, str).

Example: Immutability

```
s1 = "hello"
print("Original String:", s1)

s1[0] = "H"   # ❌ Error - strings are immutable
Instead, create a new string:
s2 = "H" + s1[1:]
print("Modified String:", s2)
Check object IDs:
print("s1 id:", id(s1))
print("s2 id:", id(s2))   # different id → new object created
```

Example: Mutable Object (list)

```
lst = ["h", "e", "l", "l", "o"]
print("Original List:", lst)

lst[0] = "H"   # ✅ works
print("Modified List:", lst)
print("lst id:", id(lst))  # same id → modified in place
```

Summary:

- String → immutable (cannot be modified in place).
- List, dict, set → mutable.

String Length

len()- returns the length of a string

```
a = "Welcome"
print(len(a))  # 7
```

String Methods

(a) Case Conversion

```
s = "hello"
print(s.capitalize())  # Hello


s = "HeLLo"
print(s.casefold())    # hello


print("Hello".lower())  # hello
print("Hello".upper())  # HELLO
print("welcome to python".title())  # Welcome To Python
print("Welcome To Python".swapcase())  # wELCOME tO pYTHON
```

(b) Alignment & Formatting

```
print("Hi".center(6, '*'))      # **Hi**
print("Hello, {}!".format("John"))  # Hello, John!
```

(c) Search & Find

```
s = "hello"
print(s.find("l"))   # 2
print(s.find("x"))   # -1
print(s.index("e"))  # 1
# print(s.index("x"))  # ❌ Error
```

(d) Count & Replace

```
s = "banana"
print(s.count("a"))   # 3
s = "Hello World"
print(s.replace("World", "There"))  # Hello There
```

(e) Validation (Boolean Checks)

```
print("abc123".isalnum())   # True
print("abc!".isalnum())     # False
print("Hello".isalpha())    # True
print("hello".islower())    # True
print("HELLO".isupper())    # True
```

(f) isdecimal(), isdigit(), isnumeric()

- **isdecimal()**
  - Returns **True** if all characters in the string are **decimal characters (0–9 only)**.
  - These are strictly digits used to form base-10 numbers.
  - Does **not** consider superscripts, fractions, or Roman numerals as valid decimals.

  Example:

  ```
  print("123".isdecimal())   # True (all decimal digits)
  print("①②③".isdecimal())   # False (circled numbers are not decimal)
  print("10.5".isdecimal())  # False (dot is not decimal)
  ```

- **isdigit()**
  - Returns **True** if all characters are **digits**.
  - Digits include **decimal digits (0–9) plus** other digit characters, such as superscripts, subscript digits, or digits from other numeral systems.

  Example:

  ```
  print("123".isdigit())     # True (decimal digits)
  print("²³".isdigit())      # True (superscript 2 and 3)
  print("⑤".isdigit())       # True (Japanese/Chinese digit in a circle)
  print("10.5".isdigit())    # False (dot not allowed)
  ```

- **isnumeric()**
  - Returns **True** if all characters are numeric.

- This is the **broadest check** — includes **digits, decimals, vulgar fractions, Roman numerals, currency numerators, etc.**
- Accepts everything isdecimal() and isdigit() accept, plus additional numeric characters.

Example:

```
print("123".isnumeric())   # True (decimal digits)
print("²³".isnumeric())    # True (superscripts)
print("⑤".isnumeric())     # True (circled digit)
print("⅔".isnumeric())     # True (fraction two-thirds)
print("Ⅷ".isnumeric())     # True (Roman numeral 8)
print("10.5".isnumeric())  # False (dot not allowed)
```

Method	Accepts Only 0–9	Superscripts (², ⑤)	Fractions & Roman (⅔, Ⅷ)
isdecimal()	✅ Yes	❌ No	❌ No
isdigit()	✅ Yes	✅ Yes	❌ No
isnumeric()	✅ Yes	✅ Yes	✅ Yes

- Use isdecimal() for base-10 digits only.
- Use isdigit() for digits + superscripts.
- Use isnumeric() for the broadest check.

(g) Splitting & Joining

```
s = "one,two,three"
print(s.split(","))   # ['one', 'two', 'three']
```

(h) Start & End Checks

```
s = "hello"
print(s.startswith("he"))  # True
s = "hello.py"
print(s.endswith(".py"))   # True
```

(i) Trimming Whitespaces

```
print("  hello  ".strip())   # hello
print("  hello".lstrip())    # hello
print("hello  ".rstrip())    # hello
```

String Methods Summary

- **Case conversion** → capitalize(), casefold(), lower(), upper(), title(), swapcase()
- **Alignment & formatting** → center(), format()
- **Search & find** → find(), index()
- **Counting & replacing** → count(), replace()
- **Validation** → isalnum(), isalpha(), isdecimal(), isdigit(), isnumeric(), islower(), isupper()
- **Splitting** → split()
- **Start/End checks** → startswith(), endswith()
- **Trimming spaces** → strip(), lstrip(), rstrip()

Python Collections

List

List

- A **list** is a collection which is:
  - **Ordered** (items have a defined order and can be accessed using indexes).
  - **Changeable (Mutable)** (we can add, update, or remove items).
  - **Duplicate values** are allowed in the list.
- Lists are written inside **square brackets [ ]**.

Creating a List

```
mylist1 = [10, 20, 30, 40, 50]

mylist2 = ["apple", "banana", "cherry"]

mylist3 = ["apple", 10, "banana", 20]  # mixed data types

mylist4 = list()   # empty list



print(mylist1)   # [10, 20, 30, 40, 50]

print(mylist2)   # ['apple', 'banana', 'cherry']
```

Accessing List Items

- Indexing starts from **0**.
- Negative indexing starts from **-1** (last element).

```
mylist = ["apple", "banana", "cherry"]

print(mylist[0])    # apple

print(mylist[-1])   # cherry
```

Range of Indexes

```
mylist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]

print(mylist[2:5])    # ['cherry', 'orange', 'kiwi']

print(mylist[-4:-1])  # ['orange', 'kiwi', 'melon']
```

Changing Item Value

```
mylist = ["apple", "banana", "cherry"]
mylist[1] = "orange"
print(mylist)  # ['apple', 'orange', 'cherry']
```

Loop Through a List

```
mylist = ["apple", "banana", "cherry"]
for item in mylist:
    print(item)
```

Check if Item Exists

```
mylist = ["apple", "banana", "cherry"]
if "apple" in mylist:
    print("Yes, apple is present")
```

List Functions

```
mylist = ["apple", "banana", "cherry"]
print(len(mylist))          # 3 (length of list)
print(mylist.count("apple"))  # count occurrences
```

Sorting & Reversing

```
mylist = ["cherry", "mango", "banana", "apple"]
mylist.sort()
print(mylist)  # ['apple', 'banana', 'cherry', 'mango']

mylist.sort(reverse=True)
print(mylist)  # ['mango', 'cherry', 'banana', 'apple']

mylist.reverse()
print(mylist)  # ['apple', 'banana', 'cherry', 'mango']
```

Note: You cannot sort heterogenous data.

```
mylist=["a", 1, "x"]
mylist.sort() #TypeError
print(mylist)
```

Adding Items

```
mylist = ["apple", "banana"]
mylist.append("cherry")        # add at end
mylist.insert(1, "orange")     # insert at specific position
print(mylist)  # ['apple', 'orange', 'banana', 'cherry']
```

Removing Items

```
mylist = ["apple", "banana", "cherry"]
mylist.remove("banana")   # remove by value
mylist.pop(0)             # remove by index
del mylist[1]             # delete specific item
mylist.clear()            # clears all items
```

Copying a List

```
mylist1 = ["apple", "banana"]
mylist2 = mylist1.copy()
print(mylist2)  # ['apple', 'banana']
```

Joining Lists

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
# Approach 1: +
print(list1 + list2)  # ['a', 'b', 'c', 1, 2, 3]
# Approach 2: extend()
list1.extend(list2)
print(list1)  # ['a', 'b', 'c', 1, 2, 3]
```

Key Point: Lists are **mutable** → you can **modify** them anytime.

Python Collections

Set

A **set** in Python is a collection that has special properties:

- **Unordered** → Items do not have a fixed position (no indexing).
- **Mutable** → We can add or remove items (but cannot access or change by index).
- **No Duplicates Allowed** → Duplicate values are automatically removed.
- Written with **curly braces { }**.

Creating a Set

```
myset1 = {10, 20, 30, 40, 50}
myset2 = {"apple", "mango", "cherry"}
myset3 = {10, "apple", "A", True}  # can store mixed data types

myset4 = set()   # empty set (use set(), {} creates a dictionary)
```

Accessing Set Items

- You **cannot access items by index** (since sets are unordered).
- You **cannot update specific items**.
- But you can **loop through** the set.

```
myset = {"apple", "banana", "cherry"}
for x in myset:
    print(x)
```

Check if Item Exists

```
myset = {"apple", "banana", "cherry"}
print("banana" in myset)   # True
print("orange" in myset)   # False
```

```
if "banana" in myset:
    print("Yes, banana exists")
```

Set Functions

```
myset = {"apple", "banana", "cherry"}
print(len(myset))   # 3 (size of the set)
```

❌ Counting duplicates → **Not possible** (since duplicates don't exist).
❌ Sorting / Reversing → **Not possible** (since set is unordered).

Adding Items

- **add()** → adds a single item
- **update()** → adds multiple items

```
myset = {"apple", "banana", "cherry"}
myset.add("orange")
print(myset)  # {'banana', 'apple', 'orange', 'cherry'}

myset.update(["mango", "grapes"])
print(myset)  # {'banana', 'apple', 'cherry', 'grapes', 'orange', 'mango'}
```

Removing Items

1. **remove(value)** → removes item, error if not found.
2. **discard(value)** → removes item, no error if not found.
3. **pop()** → removes a random item.

```
myset = {"apple", "banana", "cherry"}
myset.remove("banana")
print(myset)  # {'apple', 'cherry'}
myset.discard("xyz")  # No error even if item not present
myset.pop()  # removes random item
print(myset)
```

Clear / Delete a Set

```
myset = {"apple", "banana", "cherry"}

myset.clear()

print(myset)   # set() → empty set


del myset

# print(myset)  # NameError (set is deleted)
```

Copying a Set

```
myset1 = {"apple", "banana", "cherry"}


myset2 = myset1.copy()       # Approach 1

myset3 = set(myset1)         # Approach 2


print(myset2)

print(myset3)
```

Joining Two Sets

1. Union → Combines all items from both sets

```
set1 = {"a", "b", "c"}

set2 = {1, 2, 3}

set3 = set1.union(set2)

print(set3)  # {'a', 'b', 'c', 1, 2, 3}
```

2. Intersection → Returns only common items

```
set1 = {"apple", "banana", "cherry"}

set2 = {"google", "microsoft", "apple"}


set3 = set1.intersection(set2)

print(set3)  # {'apple'}
```

Summary

- **Set is Unordered** → No indexing, sorting, or reversing.
- **Mutable** → You can add or remove items.
- **No Duplicates** → Each item is unique.

Python Collections

(Tuple)

Tuple

- A **tuple** is a collection which is:
  - **Ordered** (items have an index).
  - **Unchangeable (Immutable)** (cannot modify, add, or remove items).
- Tuples are written inside **round brackets ( )**.

Creating a Tuple

```
mytuple = ("apple", "banana", "cherry")
print(mytuple)  # ('apple', 'banana', 'cherry')
```

Accessing Items

```
mytuple = ("apple", "banana", "cherry")
print(mytuple[1])   # banana
print(mytuple[-1])  # cherry
```

Range of Indexes

```
mytuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon")
print(mytuple[2:5])    # ('cherry', 'orange', 'kiwi')
print(mytuple[-4:-1])  # ('orange', 'kiwi', 'melon')
```

Count and Length

```
mytuple = ("apple", "banana", "apple", "cherry")
print(mytuple.count("apple"))  # 2
print(len(mytuple))          # 4
```

Loop Through a Tuple

```
mytuple = ("apple", "banana", "cherry")
for item in mytuple:
   print(item)
```

Check if Item Exists

```
mytuple = ("apple", "banana", "cherry")
if "banana" in mytuple:
   print("Yes, banana exists")
```

Workaround to Modify a Tuple

Since tuples are immutable, you can convert them to a list, modify, and convert back.

```
mytuple = ("apple", "banana", "cherry")
temp_list = list(mytuple)
temp_list[0] = "orange"
mytuple = tuple(temp_list)
print(mytuple)  # ('orange', 'banana', 'cherry')
```

Joining Tuples

```
tuple1 = ("a", "b", "c")
tuple2 = (1, 2, 3)
tuple3 = tuple1 + tuple2
print(tuple3)  # ('a', 'b', 'c', 1, 2, 3)
```

Key Point: Tuples are **immutable** → once created, they **cannot be changed**.

Summary

- **List:** Mutable (changeable), written with [ ].
- **Tuple:** Immutable (unchangeable), written with ( ).
- Use **lists** when you need to update items.
- Use **tuples** when data should stay constant.

List Vs Set Vs Tuple Vs Dictionary in Python

Feature	List	Set	Tuple	Dictionary
Definition	Ordered collection of items	Unordered collection of unique items	Ordered, immutable collection of items	Collection of key-value pairs
Syntax	list1 = [10, 20, 30]	set1 = {10, 20, 30}	tuple1 = (10, 20, 30)	dict1 = {"a": 1, "b": 2}
Order maintained?	✅ Yes	❌ No (unordered)	✅ Yes	✅ Yes (since Python 3.7+)
Indexing supported?	✅ Yes (list1[0])	❌ No	✅ Yes (tuple1[1])	✅ Keys can be accessed (dict1["a"])
Mutable (changeable)?	✅ Yes	✅ Yes (can add/remove items)	❌ No (immutable)	✅ Yes
Duplicates allowed?	✅ Yes	❌ No (all items must be unique)	✅ Yes	❌ Keys must be unique (values can repeat)
Heterogeneous data allowed?	✅ Yes	✅ Yes	✅ Yes	✅ Yes
Common use cases	Store multiple values in order, can modify	Store unique values, remove duplicates, fast membership test	Fixed collection of items (like constants)	Store data as key-value pairs (like a real-world dictionary)
Example Operations	append(), remove(), sort()	add(), remove(), union(), intersection()	Count, index (tuple.count(), tuple.index())	keys(), values(), items(), update()

In simple words:

- **List** → Ordered, changeable(mutable), allows duplicates.
- **Set** → Unordered, unique values only, mutable.
- **Tuple** → Ordered, unchangeable(immutable), allows duplicates.
- **Dictionary** → Stores data in **key-value** pairs, keys must be unique.

Python Dictionary

A **dictionary** in Python is a collection of data stored in **key : value pairs**.

Properties of a dictionary:

- **Ordered (from Python 3.7+)** – items maintain insertion order.
- **Mutable (Changeable)** – we can add, update, or remove items.
- **Indexed by keys** – values are accessed using keys (not positions).
- **No duplicate keys** – each key must be unique.
- Written using **curly braces { }**.

1. Creating a Dictionary

Approach 1: Using {}

```
mydic = {"brand": "Ford", "model": "Aspire", "year": 2024}
```

Approach 2: Using dict() constructor

```
mydic = dict(name="John", age=36, country="Norway")
```

Values can be of any data type:

```
mydic = {
  "brand": "Ford",
  "electric": True,
  "year": 2025,
  "colors": ["red", "white", "blue"]
}
```

2. Accessing Items

- **Using key name**

```
print(mydic["model"])   # Aspire
```

- **Using get() method**

```
print(mydic.get("model"))   # Aspire
```

3. Get Keys

```
print(mydic.keys())   # dict_keys(['brand', 'model', 'year'])
```

4. Get Values

```
print(mydic.values())   # dict_values(['Ford', 'Aspire', 2024])
```

5. Get Items (Key-Value Pairs)

```
print(mydic.items())   # dict_items([('brand', 'Ford'), ('model', 'Aspire'), ('year', 2024)])
```

6. Check if a Key Exists

```
if "model" in mydic:
    print("Yes, exists")
```

7. Adding Items

```
mydic["color"] = "red"
print(mydic)
# {'brand': 'Ford', 'model': 'Aspire', 'year': 2025, 'color': 'red'}
```

8. Updating Items

```
mydic.update({"color": "white"})
mydic.update({"year": 2026})
```

9. Removing Items

- **pop(key)** → removes specific key

  ```
  mydic.pop("model")
  ```

- **popitem()** → removes the last inserted item (random in <3.7)

  ```
  mydic.popitem()
  ```

- **del keyword** → removes a specific key or the entire dictionary

  ```
  del mydic["model"]   # remove key
  ```

```
del mydic          # delete dictionary
```

- **clear()** → empties dictionary

```
mydic.clear()
```

10. Copying a Dictionary

- Using **copy()**

```
mydic2 = mydic.copy()
```

- Using **dict()**

```
mydic2 = dict(mydic)
```

11. Looping Through a Dictionary

```
mydic = {"brand": "Ford", "model": "Aspire", "year": 2025}

# Loop through keys
    for key in mydic:
      print(key)
# Loop through values
    for value in mydic.values():
      print(value)
# Loop through key-value pairs
    for key, value in mydic.items():
      print(key, value)
```

Summary

- A **dictionary** stores data in **key : value** pairs.
- **Keys are unique**; values can be any type.
- **Ordered (3.7+), Mutable, Indexed by keys**.
- Supports many useful methods: .keys(), .values(), .items(), .get().
- Great for storing structured data (like JSON).

Python Functions

What is a Function?

- A **function** is a block of code that runs only when it is called.
- You can pass **data (parameters)** into a function.
- A function can **return data (output)** back to the caller.
- Functions in Python are defined using the def keyword.

Syntax

```
def function_name(parameters):
    # function body
    return value
```

- **Parameter**: variable in function definition.
- **Argument**: actual value passed when calling the function.

Function Examples

1. No parameters, no return value

```
def myfun():
    print("Hello")


myfun()   # Output: Hello
```

2. Function with parameters, no return value

```
def myfun(name):
    print("Hello", name)


myfun("Smith")   # Output: Hello Smith
```

3. Function with parameters and return value

```
def cal(a, b):
    return a + b


print(cal(10, 20))   # Output: 30
```

4. Function returning None

```
def myfunc():
    return


print(myfunc())   # Output: None
```

Types of Function Arguments

1. **Arbitrary (Variable-length) Arguments** – use *args
2. **Positional (Required) Arguments**
3. **Keyword Arguments**

Example 1: Arbitrary Arguments (*args)

If you don't know how many arguments will be passed, use * before the parameter name. It will be received as a **tuple**.

```
def sum_function(*numbers):
    total = 0
    for i in numbers:
        total += i
    return total


print(sum_function(1, 2))      # 3
print(sum_function(1, 2, 3))   # 6
print(sum_function())          # 0
```

Example 2: Positional & Keyword Arguments

```
def myfunc(i, j):
    print(i, j)


myfunc(10, 20)          # Positional arguments → 10 20
```

```
myfunc(j=20, i=10)      # Keyword arguments → 10 20
```

Example 3: Default Arguments

You can assign default values to parameters.

```
def myfunc(i, j=10):
    print(i, j)

myfunc(100, 200)   # 100 200
myfunc(100)        # 100 10
```

Example 4: Keyword Arguments

```
def greetings(name, greetmsg):
    print(greetmsg + " " + name)

greetings(name='John', greetmsg="Hello")   # Hello John
greetings(greetmsg="Hi", name='Alice')     # Hi Alice
```

Example 5: Mixing Positional & Keyword Arguments

```
def myfunc(a, b, c):
    print(a, b, c)

myfunc(10, 20, 30)            # Positional
myfunc(a=10, b=20, c=30)      # All Keyword
myfunc(10, 20, c=30)          # Mix → 10 20 30
```

⚠️ **Rule**: Positional arguments must come **before** keyword arguments.
❌ myfunc(10, b=20, 30) → Error

Example 6: Function Returning Multiple Values

```
def largest(a, b):
    if a > b:
        return a, b
    else:
        return b, a

res = largest(10, 20)
print(res)        # (20, 10)
print(type(res))  # tuple
```

Scope of Variables in Functions

- **Global Variable**: Declared outside functions, accessible everywhere.
- **Local Variable**: Declared inside a function, accessible only inside it.

Example 1: Global & Local Variables

```
x = 20  # Global

def myfunc():
    y = 10  # Local
    print(x)  # Access global
    print(y)

myfunc()
print(x)  # OK
print(y)  # Error → local variable not accessible
```

Example 2: Same Name for Global & Local

```
x = 100   # Global

def myfunc():
    x = 200   # Local
    print(x)  # Local variable used

myfunc()   # Output: 200
```

Example 3: Updating Global Variable Inside Function

```
x = 100

def myfunc():
    global x
    x = 200   # Modifies global variable
    print(x)

myfunc()    # 200
print(x)    # 200
```

Example 4: Declaring Global Variable Inside Function

```
def myfunc():
    global x
    x = 100
    print(x)

myfunc()    # 100
print(x)    # 100 (accessible globally)
```

Key Takeaways

- Use ***def*** to define functions.
- Parameters = placeholders, Arguments = actual values.
- Functions can return single or multiple values.
- Global variables are accessible everywhere, but **prefer local variables** inside functions for better practice.

Python Class, Object, Method, Static Method, and Constructor

Python is an object-oriented programming (OOP) language. The main building blocks of OOP in Python are **classes**, **objects**, **methods**, and **constructors**.

Class

- A **class** is a blueprint or template to create objects.
- It can contain:
    - **Variables** (class variables or instance variables)
    - **Methods** (functions inside a class)

Example:

```
class MyClass:
    def myfun(self):
        pass  # simple method, does nothing

    def display(self, name):
        print(name)
```

Here, **MyClass** is a class with two methods.

Object

- An **object** is an instance of a class.
- We create an object by calling the class like a function:

Example:

```
mc1 = MyClass()   # object of MyClass
mc1.display("scott")  # calling method using object
```

- Each object can access the methods and variables defined inside the class.
- You can also create **multiple objects** of the same class:

```
obj1 = MyClass()
obj2 = MyClass()
```

Instance Method

- A **method** defined inside a class that takes **self** as the first parameter.
- **self** refers to the **object** that is calling the method.
- Instance methods need an object to be called.

Example:

```
class MyClass:
    def m1(self):   # instance method
        print("this is instance method...")


mc = MyClass()
mc.m1()   # call using object
```

Static Method

- Defined inside a class using **@staticmethod**.
- Does **not need *self*** (object reference). Even if you specify **self**, that will be treated as parameter.
- Can be called using **both class name or object**.

Example:

```
class MyClass:
    @staticmethod
    def m2(x, y):
        print(x, y)


MyClass.m2(10, 20)   # called using class
mc = MyClass()
mc.m2(100, 200)      # called using object
```

Class Variables vs Instance Variables vs Local Variables

- **Class Variables**: Declared inside a class but outside methods. Shared by all objects.
- **Instance Variables**: Declared using self. inside methods. Different for each object.
- **Local Variables**: Declared inside a method (normal function variables).

Example:

```
i, j = 15, 25   # global variables
class MyClass:
    a, b = 10, 20   # class variables

    def add(self, x, y):
        print(x + y)           # local variables
        print(self.a + self.b)   # class variables
        print(i + j)           # global variables

mc = MyClass()
mc.add(100, 200)
```

Constructor (__init__)

- A **special method** in Python (__init__) that gets called automatically when an object is created.
- Used to **initialize instance variables**.
- Similar to a constructor in Java or C++.

Example:

```
class MyClass:
    def __init__(self):  # constructor
        print("this is constructor..")

    def m1(self):
        print("hello...")
```

```
mc = MyClass()   # constructor is invoked automatically
mc.m1()          # normal method must be called explicitly
```

Constructor with Parameters

Constructors can accept parameters to initialize object data.

Example:

```
class Emp:
    def __init__(self, eid, ename, sal):
        self.eid = eid
        self.ename = ename
        self.sal = sal

    def display(self):
        print(self.eid, self.ename, self.sal)

e1 = Emp(101, "john", 500000)
e1.display()   # 101 john 500000
```

__str__() Method

- A **special method** that defines how an object is represented as a string.
- When you use print(object), Python calls __str__() internally.

Example:

```
class Emp:
    def __init__(self, eid, ename, sal):
        self.eid = eid
        self.ename = ename
        self.sal = sal
```

```
    def __str__(self):
        return self.ename   # must return string only


e1 = Emp(101, "john", 500000)
print(e1)   # Output: john
```

Key Takeaways:

1. **Class** = blueprint, **Object** = real entity created from class.
2. **Instance methods** use self, need object to call.
3. **Static methods** use @staticmethod, can be called without object.
4. **Class variables** are shared across all objects; **instance variables** are unique per object.
5. **Constructor (__init__)** runs automatically when an object is created.
6. **__str__()** is used to define a string representation of an object.

Inheritance and Polymorphism in Python

What is Inheritance?

- **Inheritance** allows a class (child/derived class) to reuse properties and methods of another class (parent/base class).
- It helps in **code reusability** and **method extension**.

Types of Inheritance in Python:

1. **Single Inheritance** → One child class inherits from one parent class.
2. **Multilevel Inheritance** → A class inherits from another derived class.
3. **Hierarchical Inheritance** → Multiple child classes inherit from the same parent.
4. **Multiple Inheritance** → A child class inherits from more than one parent class.

Example 1: Basic Inheritance

```
class A:
    def m1(self):
        print("This is m1 method from class A")


class B(A):  # B inherits from A
    def m2(self):
        print("This is m2 method from class B")


bobj = B()
bobj.m1()  # Parent method
bobj.m2()  # Child method
```

- Child class B can access both its own method and parent's method.

Example 2: Single Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)


class B(A):
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)


bobj = B()
bobj.m1()  # 30
bobj.m2()  # 100
```

- Child class can access parent's variables and methods.

Example 3: Multilevel Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)


class B(A):
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)


class C(B):
    i,j = 5,2
```

```
    def m3(self):
        print(self.i * self.j)


cobj = C()
cobj.m1()  # 30
cobj.m2()  # 100
cobj.m3()  # 10
```

- Chain of inheritance: A → B → C.

Example 4: Hierarchical Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)


class B(A):
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)


class C(A):
    i,j = 5,2
    def m3(self):
        print(self.i * self.j)


bobj = B()
bobj.m1()  # 30
bobj.m2()  # 100
```

```
cobj = C()
cobj.m1()  # 30
cobj.m3()  # 10
```

- Both B and C share the same parent A.

Example 5: Multiple Inheritance

```
class A:
    x,y = 10,20
    def m1(self):
        print(self.x + self.y)


class B:
    a,b = 200,100
    def m2(self):
        print(self.a - self.b)


class C(A,B):   # Inherits from both A & B
    i,j = 5,2
    def m3(self):
        print(self.i * self.j)


cobj = C()
cobj.m1()  # 30
cobj.m2()  # 100
cobj.m3()  # 10
```

- C can access features of both A and B.

Example 6: Using super()

```
class A:
    def m1(self):
        print("This is m1 method from class A.....")


class B(A):
    def m1(self):
        print("This is m1 method from class B.....")
        super().m1()  # calling parent method


bobj = B()
bobj.m1()
```

- super() is used to call the **parent class method** from the child class.

Example 7: Accessing Parent Variables

```
class A:
    a,b = 10,20


class B(A):
    i,j = 100,200
    def m(self, x, y):
        print(x+y)          # Local variables
        print(self.i+self.j) # Child class variables
        print(self.a+self.b) # Parent class variables


bobj = B()
bobj.m(1000,2000)
```

- A child class can access local variables, its own variables, and parent's variables.

What is Polymorphism?

- **Polymorphism** means "many forms".
- In Python, it allows the **same method name** to behave differently based on context.

Types of Polymorphism:

1. **Method Overriding** → Child class provides its own version of a method from the parent.
2. **Method Overloading** → Same method name but with different numbers of parameters (Python does not support true overloading but can simulate it using default arguments).

Example 8: Variable Overriding

```
class Parent:
    name = "scott"


class Child(Parent):
    name = "John"  # overriding variable


childobj = Child()
print(childobj.name)  # John
```

- Child class variable overrides parent class variable.

Example 9: Method Overriding

```
class Bank:
    def rateOfInterest(self):
        return 0


class XBank(Bank):
    def rateOfInterest(self):
        return 10.5
```

```
class YBank(Bank):
    def rateOfInterest(self):
        return 12.5


x = XBank()
print(x.rateOfInterest())  # 10.5


y = YBank()
print(y.rateOfInterest())  # 12.5
```

- Same method rateOfInterest() behaves differently in each child class.

Example 10: Method Overloading (Simulated)

```
class Human:
    def sayHello(self, name=None):
        if name is not None:
            print("Hello " + name)
        else:
            print("Hello")


h = Human()
h.sayHello()        # Hello
h.sayHello("David")  # Hello David
```

- Python achieves method overloading by using **default arguments**.

Example 11: Another Overloading Example

```
class Calculation:
    def add(self, a=0, b=0, c=0):
        print(a + b + c)
```

```
calobj = Calculation()
calobj.add()            # 0
calobj.add(10, 20)      # 30
calobj.add(100,200,300)  # 600
```

- Same method works with different number of arguments.

Summary

- **Inheritance** → Reuse parent class methods/variables.
  - Types: Single, Multilevel, Hierarchical, Multiple.
  - Use super() to access parent methods/variables.
- **Polymorphism** → Same name, different behaviors.
  - Variable overriding.
  - Method overriding (child redefines parent method).
  - Method overloading (using default arguments).

Python Modules

What is a Module?

- A **module** in Python is simply a **Python file (.py)** that contains functions, classes, or variables.
- Modules help in **organizing code** and **reusing it** across multiple programs.
- Example: If you have math.py with functions, you can use them in another file with import math.

✅ **Example 1: Using Functions from a Module**

📁 **Files:**

operations.py

```
def add(num1, num2):
    print(num1 + num2)
def mul(num1, num2):
    print(num1 * num2)
```

client.py

```
# Approach 1: Import whole module
        import operations
        operations.add(10, 20)
        operations.mul(10, 20)
# Approach 2: Import specific functions
        from operations import add, mul
        add(10, 20)
        mul(10, 20)
# Approach 3: Import everything
        from operations import *
        add(10, 20)
        mul(10, 20)
```

Key Points:

- import module_name → use with module_name.function().
- from module_name import func1, func2 → directly call the functions.
- from module_name import * → imports all functions, but may cause conflicts if two modules have functions with the same name.

✅ **Example 2: Same Function Names in Two Modules**

📁 **Files:**

animal.py

```
def fly():
    print("Animal Cant fly")
def color():
    print("Animal is Black")
```

bird.py

```
def fly():
    print("Bird Can Fly")
def color():
    print("Bird is Green")
```

main.py

```
# Approach 1: Import modules separately
        import animal
        import bird

        animal.fly()
        animal.color()

        bird.fly()
        bird.color()
```

Approach 2: Import everything from both

```
from animal import *
fly()
color()

from bird import *
fly()
color()
```

Key Points:

- If two modules have functions with the **same name**, importing with from module import * may cause confusion.
- To avoid conflicts, prefer **approach 1** (use module_name.function()).

✅ **Example 3: Classes in Different Modules**

📂 **Files:**

a.py

```
class Animal:
   def display(self):
      print("I like Cow")
```

b.py

```
class Bird:
   def display(self):
      print("I like Parrot")
```

main.py

```
# Approach 1: Import modules
import a
import b
obj1 = a.Animal()
obj1.display()
```

```
obj2 = b.Bird()
obj2.display()

# Approach 2: Import classes directly
from a import Animal
from b import Bird

obj3 = Animal()
obj3.display()

obj4 = Bird()
obj4.display()
```

Key Points:

- You can import entire modules or specific classes.
- If only one or two classes are needed, **direct import is cleaner**.
- If many functions/classes are needed, import the full module.

Python Packages

What is a Package?

- A **package** in Python is a **collection of modules** (Python files).
- A module = one .py file, while a package = **a folder** containing multiple .py files.
- Packages help in **organizing related modules** together, especially in large projects.
- To make a folder a package, it usually contains an __init__.py file (even empty).

✅ Example 1: Importing Modules from a Single Package

📂 Project Structure

```
pack1/
 ├── module1.py   → display()
 ├── module2.py   → show()
 └── client.py
```

module1.py

```
def display():
    print("This is display function from module1")
```

module2.py

```
def show():
    print("This is show function from module2")
```

client.py

```
import sys
sys.path.append("C:/Automation/PWPython/pack1")

# Approach 1: Import whole modules
    import module1, module2
    module1.display()
    module2.show()
```

Approach 2: Import functions directly

```
from module1 import *
from module2 import *
display()
show()
```

Key Points:

- sys.path.append(path) tells Python **where to find the package**.
- You can either **import whole modules** or **specific functions**.

✅ **Example 2: Importing from Two Different Packages**

📁 Project Structure

```
pack1/
 ├── module1.py   → display()
 └── pack2/
      └── module2.py  → show()
```

client.py

```
import sys
sys.path.append("C:/Automation/PWPython/pack1") # Import from pack1
from module1 import *
sys.path.append("C:/Automation/PWPython/pack1/pack2") # Import from pack2 inside pack1
from module2 import *
display()
show()
```

Key Points:

- We can import modules from **nested packages (sub-packages)**.
- Each sub-package acts like a mini folder with its own modules.

✅ **Example 3: Importing Classes from Different Packages**

📁 Project Structure

```
pack1/
 └── emp.py    → Employee class
pack2/
 └── stu.py    → Student class
pack3/
 └── client.py
```

emp.py

```
class Employee:
    def __init__(self, eid, ename, sal):
        self.eid = eid
        self.ename = ename
        self.sal = sal

    def displayemp(self):
        print(f"empid:{self.eid} empname:{self.ename} empsal:{self.sal}")
```

stu.py

```
class Student:
    def __init__(self, sid, sname, sgrad):
        self.sid = sid
        self.sname = sname
        self.sgrad = sgrad

    def displaystu(self):
        print(f"stuid:{self.sid} stuname:{self.sname} stusal:{self.sgrad}")
```

client.py

```
import sys


# Import Employee class
sys.path.append("C:/Automation/PWPython/pack1")
from emp import Employee


e = Employee(101, 'Scott', 40000)
e.displayemp()


# Import Student class
sys.path.append("C:/Automation/PWPython/pack2")
from stu import Student


s = Student(141, 'David', 'A')
s.displaystu()
```

Key Points:

- Classes can be **imported from different packages** into one client program.
- You can use sys.path.append() if the packages are not in the default Python path.
- This way, large projects remain **well-structured and modular**.

Summary

1. **Module** = a single .py file.
2. **Package** = a folder containing multiple modules.
3. You can import:
    - Whole modules (import module)
    - Specific functions/classes (from module import func/class)
    - Everything (from module import *, not recommended).
4. sys.path.append("path") → helps Python locate packages stored in custom folders.

Encapsulation in Python

What is Encapsulation?

- **Encapsulation** means **wrapping variables (data) and methods (functions) into a single unit (class)**.
- It also means **restricting direct access to some variables** so that they cannot be modified accidentally.
- In Python, we achieve this using **access modifiers**.

Why do we need Encapsulation?

1. **Data protection** – Prevents accidental modification of important data.
2. **Controlled access** – We decide how the data should be read or modified (using getters and setters).
3. **Security** – Hides the implementation details from the outside world.

Access Modifiers in Python

1. **Public** (default) → can be accessed anywhere.

```
self.name   # public
```

2. **Protected** (convention, single underscore) → should not be accessed directly outside the class.

```
self._marks   # protected
```

3. **Private** (double underscore) → cannot be accessed directly outside the class.

```
self.__balance   # private
```

Example of Encapsulation

```
class Student:
    def __init__(self, name):
        self.name = name
        self.__marks = 0   # private variable
```

```
    # Getter method
    def get_marks(self):
        return self.__marks

    # Setter method
    def set_marks(self, marks):
        if 0 <= marks <= 100:
            self.__marks = marks
        else:
            print("Invalid marks! Must be between 0 and 100.")

# Usage
s1 = Student("Rahul")
s1.set_marks(90)        # setting value through setter
print(s1.get_marks())   # getting value through getter
```

Abstraction in Python

Data abstraction means showing only the essential features and hiding the complex internal details.

In Python abstraction is used to **hide the implementation details** from the user and expose only necessary parts.

Abstract Base Class (ABC)

- In Python, an Abstract Base Class (ABC) is used to achieve data abstraction by defining a common interface for its subclasses.
- It cannot be instantiated directly and serves as a blueprint for other classes.

Example:

```
from abc import ABC, abstractmethod

class Greet(ABC):
    @abstractmethod
    def say_hello(self):
        pass  # Abstract method

class English(Greet):
    def say_hello(self):
        return "Hello!"

g = English()
print(g.say_hello())
```

Explanation:

- Greet is an abstract class with a method **say_hello()** that has no implementation.
- English implements this method and returns a greeting.
- This keeps structure fixed while letting subclasses define their own behavior.

Abstract Class Instantiation

- Abstract classes **cannot be instantiated directly**. This is because they contain one or more abstract methods or properties that lack implementations.
- Attempting to instantiate an abstract class results in a *TypeError*.

Python File Handling

File handling in Python allows us to **create, read, write, update, and delete files and directories**.
The open() function is the most commonly used to work with files.

Opening a File

file = open("filename.txt", "mode")

File Modes:

- 'r' → Read (default). File must exist.
- 'w' → Write. Creates a new file if not exists, **overwrites** existing content.
- 'a' → Append. Creates a new file if not exists, **adds** content at the end without overwriting.
- 'x' → Create. Creates a new file but raises an error if the file already exists.

Writing to a File

Example: Using open() with w mode

```
file = open("C:\\Automation\\automationFiles\\myfile.txt", 'w')
file.write("Welcome to Python \nFile Handling")
file.close()
```

Example: Using *with* statement

```
with open("C:\\Automation\\automationFiles\\myfile.txt", 'w') as file:
    file.write("Welcome to Python \nFile Handling")
```

- The with statement automatically **closes** the file after use.

Appending Data to a File

```
file = open("C:\\Automation\\automationFiles\\myfile.txt", 'a')
file.write("\nThis line is appended.")
file.close()
```

Note: Unlike write mode (w), append mode (a) **does not delete old content**.

Reading from a File

There are multiple ways to read:

```
file = open("C:\\Automation\\automationFiles\\myfile.txt", 'r')
content = file.read()      # Reads the entire file
print(content)
```

 # file.readline() → Reads one line at a time

file.readlines() → Reads all lines into a list

file.close()

Renaming a File

```
import os
sourcefile = "C:\\Automation\\automationFiles\\myfile.txt"
targetfile = "C:\\Automation\\automationFiles\\myfile1.txt"

os.rename(sourcefile, targetfile)
print("File renamed")
```

Deleting a File

```
import os

file = "C:\\Automation\\automationFiles\\myfile1.txt"
if os.path.exists(file):
   os.remove(file)
else:
   print("File does not exist")
```

Creating a Directory

```
import os
os.mkdir("C:\\Automation\\automationFiles\\mydir")
print("Directory created")
```

Checking if a Directory Exists

```
import os
dir_name = "C:\\Automation\\automationFiles\\mydir"

if os.path.exists(dir_name):
   print("Directory exists")
else:
   print("Directory does not exist")
```

Renaming a Directory

```
import os

sourcedir = "C:\\Automation\\automationFiles\\mydir"
targetdir = "C:\\Automation\\automationFiles\\mydir1"

os.rename(sourcedir, targetdir)
print("Directory renamed")
```

Removing a Directory

```
import os

os.rmdir("C:\\Automation\\automationFiles\\mydir1")
print("Directory removed")
```

Note:

os.rmdir() only removes **empty directories**.

For non-empty folders, use:

```
import shutil
shutil.rmtree("folder_name")
```

Get Current Working Directory

```
import os

current_directory = os.getcwd()
print(f"Current working directory: {current_directory}")
```

Python Exceptions

Difference Between Error and Exception

Error

- Errors are serious issues in the program that **cannot be handled at runtime**.
- They usually occur because of mistakes in the code's syntax or configuration.
- Errors must be **fixed by the programmer**.
- Examples: **SyntaxError, MemoryError**

🔹 Example:

```
print("Hello world"   # ❌ Missing closing parenthesis → SyntaxError
```

Exception

- Exceptions are issues that occur **while the program is running**.
- They are **less severe than errors** and can be handled by the program.
- Examples: dividing by zero, invalid input, file not found, etc.
- Handling exceptions allows the program to **continue running** without crashing.

🔹 Example:

```
n = 10
res = n / 0  # ❌ ZeroDivisionError (Exception)
```

Exception Handling in Python

Python provides try, except, else, and finally blocks to handle exceptions.

1. try-except Block

The code inside the try block is executed.
If an exception occurs, it jumps to the except block.

```
try:
    print(x)   # x is not defined → NameError
except:
    print("An exception occurred")
```

2. Handling Multiple Exceptions

You can define **specific exception types** in separate except blocks.

```
try:
    print(x)
except NameError:
    print("Variable x is not defined")
except:
    print("Something else went wrong")
```

3. try-except-else

- The else block runs **only if no exception occurs**.

```
try:
    print("Hello")
except:
    print("Something went wrong")
else:
    print("Nothing went wrong")
```

4. try-except-finally

- The finally block **always executes**, no matter what.
- Useful for cleanup operations like closing files or releasing resources.

```
try:
    print(x)  # x is not defined
except:
    print("Something went wrong")
finally:
    print("The 'try except' is finished")
```

5. try-except-else-finally Together

All four blocks can be combined.

```
try:
    n = int(input("Enter a value: "))
    res = 100 / n
except ZeroDivisionError:
    print("You can't divide by zero!")
except ValueError:
    print("Enter a valid number!")
else:
    print("Result is", res)
finally:
    print("Execution complete.")
```

6. Nested try Blocks

A try block can exist inside another try block.

```
try:
    file = open("C:/Automation/automationFiles/myfile.txt", "w")
    try:
        file.write("Welcome")
    except:
        print("Something went wrong when writing to the file")
    finally:
        file.close()
except:
    print("Something went wrong when opening the file")
```

Raising Exceptions

Sometimes you may want to **manually raise an exception** when certain conditions are not met.

1. raise Exception

```
x = -1

if x < 0:
   raise Exception("Sorry, no numbers below zero")
```

2. raise Specific Exception

You can raise a particular type of exception.

```
x = "hello"

if not type(x) is int:
   raise TypeError("Only integers are allowed")
```

User-Defined Exceptions

You can raise your own custom exceptions to handle specific situations.

```
def set(age):
   if age < 0:
      raise ValueError("Age cannot be negative.")
   print(f"Age set to {age}")


# usage
try:
   set(-5)
except ValueError as e:
   print(e)
```

Key Takeaways

1. **Errors** → Serious, must be fixed in code.
2. **Exceptions** → Runtime issues that can be handled.
3. Use **try-except** to catch exceptions.
4. **else** runs if no error occurs.
5. **finally** always runs (cleanup).
6. You can **raise exceptions** manually.
7. **User-defined exceptions** improve error handling in custom cases.